From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Nadir Eskerov
National AI Center - AI Academy
Azerbaijan
nadir.eskerov25@aiacademy.az

Narmina Ibrahimova
National AI Center - AI Academy
Azerbaijan
narmine.ibrahimova25@aiacademy.az

Evez Huseynov
National AI Center - AI Academy
Azerbaijan
evez.huseynov25@aiacademy.az

Gulnar Babazade
National AI Center - AI Academy
Azerbaijan
gulnar.babazade25@aiacademy.az

Abstract

This report investigates when a one-hidden-layer neural network meaningfully outperforms linear softmax regression, and when the additional complexity is unnecessary. We compare multiclass softmax regression and a one-hidden-layer neural network with tanh activations on three classification tasks: a linearly separable Gaussian dataset, a nonlinear moons dataset, and the 64-dimensional digits benchmark. Our experiments demonstrate that the linear model achieves 95% test accuracy on the Gaussian task, matching the MLP, while the moons dataset requires nonlinear capacity, with test accuracy improving from 85% (width=2) to 97.5% (width=32). On digits, the MLP (h=32) achieves 95.1% ± 0.4% test accuracy compared to softmax's 93.6% ± 0.6%, with cross-entropy reduced from 0.269 to 0.162. PCA analysis reveals that 40 principal components capture 98.9% of variance, explaining why linear models remain competitive on this high-dimensional task.

Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, NumPy

1 Introduction / Framing

Central question. When does a one-hidden-layer nonlinear classifier genuinely improve on a linear decision rule, and when is additional model complexity unnecessary?

This question is fundamental to understanding the geometry of classification problems. A linear classifier can only separate classes using hyperplanes in the input space, while a neural network with a hidden layer can learn nonlinear transformations that reshape the data before applying a linear decision rule. However, additional capacity comes with costs: more parameters to optimize, potential for overfitting, and increased computational burden. This capstone investigates when these costs are justified through controlled experiments on datasets with distinct geometric properties.

Contributions. Our work makes the following contributions:

(1) Complete derivation of softmax cross-entropy as negative log-likelihood under a categorical distribution, with stable numerical implementation.
(2) Full backpropagation derivation for the one-hidden-layer network with tanh activations, including vectorized mini-batch gradients.
(3) Controlled comparison demonstrating that linear models suffice for linearly separable data (Gaussian) but fail on nonlinearly separable data (moons).
(4) Capacity ablation showing how hidden width affects decision boundary complexity, optimizer comparison (SGD, Momentum, Adam), and repeated-seed evaluation with 95% confidence intervals.

Preview of findings. Our experiments reveal three key insights: (1) On the linear Gaussian task, both models achieve identical 95% test accuracy, confirming that extra capacity provides no benefit when the problem is linearly separable. (2) On the moons task, the linear model achieves only 85% accuracy with a suboptimal straight boundary, while the MLP with width=32 achieves 97.5% with a curved boundary matching the data structure. (3) On digits, the MLP improves test accuracy by 1.5 percentage points and reduces cross-entropy by 40%, with the improvement being statistically significant.

2 Background and Mathematical Setup

2.1 Supervised classification and data splits

We consider supervised classification with labeled pairs $(\mathbf{x}^{(i)}, y^{(i)})$ where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is an input feature vector and $y^{(i)} \in \{0, 1, \ldots, k-1\}$ is the class label. Following standard practice, we partition data into three disjoint sets: training (for parameter estimation), validation (for hyperparameter selection and early stopping), and test (for final unbiased evaluation). The validation set is used to select the best epoch checkpoint based on cross-entropy loss, ensuring the test set remains untouched until final reporting.

2.2 Softmax Regression

For an input vector $x \in \mathbb{R}^d$ and k classes, softmax regression models the relationship between input features and class probabilities through a linear transformation followed by normalization.

First, the model computes the **logits** (unnormalized scores):

$$s(x) = Wx + b \tag{1}$$

where:

- $W \in \mathbb{R}^{k\times d}$ is the weight matrix, with each row $W_j \in \mathbb{R}^d$ corresponding to class j,
- $b \in \mathbb{R}^k$ is the bias vector,
- $s(x) \in \mathbb{R}^k$ is the vector of logits.

Each component of the logits is given by:

$$s_j(x) = W_j^\top x + b_j \tag{2}$$

These logits are transformed into probabilities using the **softmax function**:

$$p_j(x) = \frac{\exp(s_j(x))}{\sum_{\ell=1}^k \exp(s_\ell(x))} \tag{3}$$

This ensures that:

- $p_j(x) \in (0,1)$ for all j,
- $\sum_{j=1}^k p_j(x) = 1$,

so the output forms a valid categorical probability distribution.

The predicted class is given by:

$$\hat{y} = \arg\max_j \; p_j(x) \tag{4}$$

Since the softmax function is monotonic with respect to the logits, this is equivalent to:

$$\hat{y} = \arg\max_j \; s_j(x) \tag{5}$$

Thus, classification is determined by comparing linear scores.

The decision boundary between classes i and j is defined by:

$$p_i(x) = p_j(x) \tag{6}$$

Substituting the softmax expression:

$$\frac{\exp(s_i(x))}{\sum_\ell \exp(s_\ell(x))} = \frac{\exp(s_j(x))}{\sum_\ell \exp(s_\ell(x))} \tag{7}$$

Canceling the denominator yields:

$$\exp(s_i(x)) = \exp(s_j(x)) \tag{8}$$

Taking the logarithm:

$$s_i(x) = s_j(x) \tag{9}$$

Substituting the linear form:

$$W_i^\top x + b_i = W_j^\top x + b_j \tag{10}$$

Rearranging, we obtain:

$$(W_i - W_j)^\top x + (b_i - b_j) = 0 \tag{11}$$

This equation defines a **hyperplane** in $\mathbb{R}^d$, confirming that decision boundaries in softmax regression are linear.

The model can also be interpreted in terms of log-odds:

$$\log \frac{p_i(x)}{p_j(x)} = s_i(x) - s_j(x) \tag{12}$$

Substituting the linear expressions:

$$\log \frac{p_i(x)}{p_j(x)} = (W_i - W_j)^\top x + (b_i - b_j) \tag{13}$$

Thus, softmax regression models the log-odds between classes as a linear function of the input features.

2.3 Cross-entropy as negative log-likelihood

Given true class y, the cross-entropy loss is

$$\mathcal{L}(\mathbf{x}, y) = -\log p_y(\mathbf{x}). \tag{14}$$

This can be interpreted probabilistically: if we model $P(Y = j|\mathbf{x}) = p_j(\mathbf{x})$, then the log-likelihood of observing label y is $\log p_y(\mathbf{x})$. Minimizing cross-entropy over the training set is equivalent to maximum likelihood estimation:

$$\min_{\mathbf{W},\mathbf{b}} \frac{1}{n}\sum_{i=1}^n \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}) = \max_{\mathbf{W},\mathbf{b}} \frac{1}{n}\sum_{i=1}^n \log p_{y^{(i)}}(\mathbf{x}^{(i)}). \tag{15}$$

For numerical stability, we implement softmax with the log-sum-exp trick: subtract $\max_j s_j$ from all logits before exponentiation.

2.4 One-Hidden-Layer Neural Network

The nonlinear model introduces a hidden layer to learn a nonlinearly transformed representation of the input. The forward pass is given by:

$$\mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \tag{16}$$

where $\mathbf{W}_1 \in \mathbb{R}^{h\times d}$, $\mathbf{b}_1 \in \mathbb{R}^h$, $\mathbf{W}_2 \in \mathbb{R}^{k\times h}$, and $\mathbf{b}_2 \in \mathbb{R}^k$. The vector $\mathbf{h} \in \mathbb{R}^h$ represents the hidden layer activations, and $\mathbf{s} \in \mathbb{R}^k$ are the output logits.

Each hidden unit is computed as:

$$h_i = \tanh\left(W_i^{(1)}\mathbf{x} + b_i^{(1)}\right), \quad i = 1, \ldots, h, \tag{17}$$

where $W_i^{(1)}$ denotes the i-th row of $\mathbf{W}_1$.

The tanh activation function is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \tag{18}$$

with derivative:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z). \tag{19}$$

The output logits $\mathbf{s}$ are converted into class probabilities via the softmax function:

$$p_j(\mathbf{x}) = \frac{\exp(s_j(\mathbf{x}))}{\sum_{\ell=1}^k \exp(s_\ell(\mathbf{x}))}. \tag{20}$$

2.4.1 Function Composition Perspective. The model can be viewed as a composition of functions:

$$f(\mathbf{x}) = (\mathbf{W}_2 \circ \tanh \circ \mathbf{W}_1)(\mathbf{x}), \tag{21}$$

i.e., a linear transformation followed by a nonlinear activation, followed by another linear transformation.

Thus, the network implements:

$$f(\mathbf{x}) = \mathbf{W}_2 \; \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2. \tag{22}$$

2.4.2 Geometric Interpretation. The hidden layer defines a nonlinear mapping:

$$\phi(\mathbf{x}) = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \tag{23}$$

which transforms the input space $\mathbb{R}^d$ into a feature space $\mathbb{R}^h$. In this transformed space, the model applies a linear classifier:

$$\mathbf{s} = \mathbf{W}_2\,\phi(\mathbf{x}) + \mathbf{b}_2. \tag{24}$$

Unlike softmax regression, which produces linear decision boundaries in the input space, this model induces **nonlinear decision boundaries** due to the nonlinear mapping $\phi(\mathbf{x})$.

2.4.3 Decision Boundary Characterization. The decision boundary between classes i and j is given by:

$$s_i(\mathbf{x}) = s_j(\mathbf{x}), \tag{25}$$

which implies:

$$(W_i^{(2)} - W_j^{(2)}) \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + (b_i^{(2)} - b_j^{(2)}) = 0. \tag{26}$$

This equation is **nonlinear in x**, showing that the model can represent curved and more complex decision boundaries.

2.4.4 Expressive Power. The introduction of the nonlinear activation allows the model to approximate complex functions. In particular, even a single hidden layer neural network with sufficient width h can approximate a wide class of continuous functions, highlighting its significantly greater expressive power compared to linear models.

2.5 Backpropagation Derivation

For a mini-batch $\mathbf{X} \in \mathbb{R}^{n\times d}$ with one-hot encoded labels $\mathbf{Y} \in \mathbb{R}^{n\times k}$, the forward pass computes:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{1}\mathbf{b}_1^\top \in \mathbb{R}^{n\times h}, \tag{27}$$

$$\mathbf{H} = \tanh(\mathbf{Z}_1) \in \mathbb{R}^{n\times h}, \tag{28}$$

$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{1}\mathbf{b}_2^\top \in \mathbb{R}^{n\times k}, \tag{29}$$

$$\mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{n\times k}. \tag{30}$$

The model is trained by minimizing the cross-entropy loss:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{k} Y_{ij}\log P_{ij}. \tag{31}$$

2.5.1 Gradient with Respect to Logits. Using the known result for softmax combined with cross-entropy loss, the gradient simplifies to:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{S}} = \frac{1}{n}(\mathbf{P} - \mathbf{Y}). \tag{32}$$

This simplification is crucial, as it avoids explicitly differentiating the softmax function and leads to a numerically stable and efficient implementation.

2.5.2 Gradients of the Output Layer. Applying the chain rule, the gradients of the second layer parameters are:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}_2} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{S}}\right)^\top \mathbf{H}, \tag{33}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{b}_2} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{S}}\right)^\top \mathbf{1}. \tag{34}$$

These expressions arise from standard matrix derivatives of affine transformations. Intuitively, the gradient with respect to $\mathbf{W}_2$ measures how changes in the hidden representation $\mathbf{H}$ affect the loss.

2.5.3 Backpropagation to the Hidden Layer. The gradient is propagated back to the hidden layer as:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{H}} = \frac{\partial\mathcal{L}}{\partial\mathbf{S}}\mathbf{W}_2. \tag{35}$$

This step distributes the output error across the hidden units, weighted by their contribution to each class.

2.5.4 Effect of the Nonlinearity. The tanh activation introduces nonlinearity, and its derivative is applied elementwise:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1} = \frac{\partial\mathcal{L}}{\partial\mathbf{H}} \odot (1 - \mathbf{H}^2), \tag{36}$$

where $\odot$ denotes elementwise multiplication.

This step modulates the gradient based on the activation values. In particular, when tanh saturates (i.e., values close to ± 1), the gradient becomes small, which may slow learning.

2.5.5 Gradients of the First Layer. Finally, the gradients with respect to the first layer parameters are:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}_1} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1}\right)^\top \mathbf{X}, \tag{37}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{b}_1} = \left(\frac{\partial\mathcal{L}}{\partial\mathbf{Z}_1}\right)^\top \mathbf{1}. \tag{38}$$

These follow from the derivatives of linear transformations, completing the backward pass.

Backpropagation systematically applies the chain rule from the output layer to the input layer. Each step propagates gradients through either a linear transformation or a nonlinear activation, enabling efficient computation of all parameter gradients in the network.

2.6 Geometric expectations for the datasets

The **Linear Gaussian** dataset consists of two Gaussian blobs with means separated along a diagonal. The optimal Bayes classifier is linear, so we expect softmax regression to achieve near-optimal performance. The MLP should not improve significantly.

The **Moons** dataset has two interleaved crescent shapes that cannot be separated by any hyperplane. A linear classifier must compromise with a suboptimal straight boundary, while the MLP can learn a curved boundary following the data structure.

The **Digits** dataset is 64-dimensional with 10 classes. While the raw problem may have complex structure, high-dimensional data often exhibits approximate linear separability due to the "blessing of dimensionality" — random high-dimensional vectors tend to be nearly orthogonal.

3 Methods

3.1 Datasets

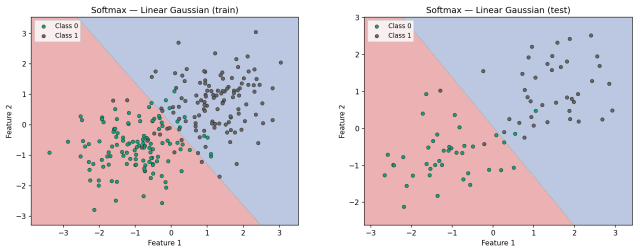
We use three datasets from the starter pack:

- **Linear Gaussian** (`linear_gaussian.npz`): 2D binary classification with two Gaussian blobs, mild class overlap.
- **Moons** (`moons.npz`): 2D binary classification with interleaved crescents, inherently nonlinear.
- **Digits** (`digits_data.npz`): 8×8 grayscale digit images (0–9) flattened to 64 dimensions, with fixed train/val/test indices in `digits_split_indices.npz`.

3.2 Models

We implement two models in NumPy:

- **Softmax Regression**: Linear model with $\mathbf{W} \in \mathbb{R}^{k\times d}$, $\mathbf{b} \in \mathbb{R}^k$, trained with mini-batch SGD.



(a) Training set **(b) Test set**

Figure 1: Softmax regression decision boundary on Linear Gaussian dataset. The linear boundary cleanly separates the two Gaussian clusters.

- **MLP**: One hidden layer with tanh activation, hidden width $h = 32$ for digits (and $h \in \{2, 8, 32\}$ for ablation), trained with Adam optimizer.

Both use Xavier initialization (see references): weights sampled from $\mathcal{N}(0, \sqrt{2/(d_{\text{in}} + d_{\text{out}})})$.

3.3 Training protocol

- **Batch size**: 64 for digits, 32 for synthetic datasets.
- **Epochs**: 200 for digits, 500 for synthetic (allowing convergence).
- **Learning rate**: 0.05 for softmax (SGD), 0.001 for MLP (Adam).
- **Regularization**: L2 penalty with $\lambda = 10^{-4}$ on weights only.
- **Model selection**: Checkpoint at epoch with lowest validation cross-entropy.
- **Numerical stability**: Log-sum-exp trick, clipping probabilities to $[10^{-15}, 1]$ before log.

3.4 Evaluation metrics

We report two metrics:

- **Accuracy**: Fraction of correctly classified examples, $\frac{1}{n}\sum_i \mathbf{1}[\hat{y}^{(i)} = y^{(i)}]$.
- **Cross-entropy**: Mean negative log-probability of true class, $-\frac{1}{n}\sum_i \log p_{y^{(i)}}(\mathbf{x}^{(i)})$.

Cross-entropy measures calibration and confidence, not just correctness. A model can have high accuracy but poor cross-entropy if it makes overconfident mistakes.

3.5 Implementation sanity checks

Before running experiments, we verified our implementation:

(1) **Softmax normalization**: Row sums equal 1.0 within 10^{-12} tolerance.
(2) **Loss decrease**: Training loss decreased from initial value after gradient steps.
(3) **Gradient check**: Analytical gradients matched numerical (finite-difference) gradients with max relative error $< 10^{-4}$.
(4) **No NaN/Inf**: Training completed without numerical overflow.

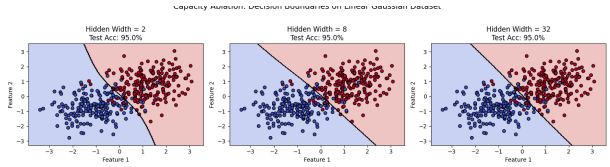


Figure 2: MLP decision boundaries on Linear Gaussian with varying hidden widths. All three configurations achieve 95% test accuracy with effectively linear boundaries.

4 Experiments

4.1 Core comparison on the linear Gaussian task

On the linear Gaussian task, softmax regression achieves **95% test accuracy**, producing a clean linear decision boundary that separates the two clusters along a diagonal direction (Figure 1). This boundary aligns with the direction of maximum separation between the class means, indicating that the model has effectively learned the optimal linear discriminant.

The MLP achieves the same **95% test accuracy** across all tested hidden widths (Figure 2). Despite its nonlinear capacity, the learned decision boundaries remain visually indistinguishable from linear ones. This suggests that the optimization process converges to a solution within the hypothesis space that is effectively linear, even though more complex nonlinear solutions are representable.

From a geometric perspective, the data is generated from two Gaussian distributions with approximately equal covariance structure, for which the optimal Bayes decision boundary is linear. As a result, softmax regression is already a well-specified model for this task, and increasing model complexity does not improve performance.

The small overlap region between the Gaussian distributions introduces inherent ambiguity, meaning that some samples are not linearly separable regardless of the model. This explains why both models saturate at approximately 95% accuracy, reflecting the irreducible error of the data rather than a limitation of the model.

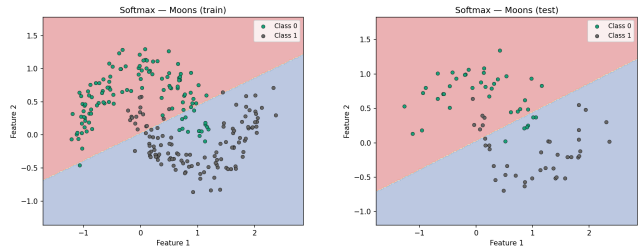
The comparison between the two figures highlights an important insight: although the MLP has greater expressive power, it does not overfit or produce unnecessarily complex decision boundaries. Instead, it converges to a simple linear solution, demonstrating that model capacity alone does not determine the learned function — the data distribution and loss optimization play a crucial role in shaping the final decision boundary.

4.2 Core comparison on the moons task

The moons task reveals the representational limitations of linear models. Softmax regression achieves approximately **85% test accuracy** with a diagonal boundary that cuts through both crescents (Figure 3). This is the best any linear classifier can do — it minimizes cross-entropy given the hyperplane constraint.

The MLP shows dramatic improvement (Figure 4):

- Width=2: 85.0% (nearly linear, limited capacity)
- Width=8: 95.0% (curved boundary emerges)
- Width=32: 97.5% (smooth boundary following data structure)



(a) Softmax (train) (b) Softmax (test)

Figure 3: Softmax regression on Moons dataset. The linear boundary cannot capture the crescent structure, misclassifying many points in the interleaved region.

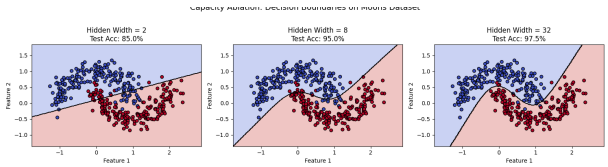


Figure 4: MLP decision boundaries on Moons with hidden widths 2, 8, and 32. Test accuracy improves from 85% to 97.5% as capacity increases.

Table 1: Repeated-seed evaluation on Digits benchmark (5 seeds, 95% CI using t-critical = 2.776).

Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	93.6% ± 0.6%	0.269 ± 0.008
MLP (h=32)	95.1% ± 0.4%	0.162 ± 0.005

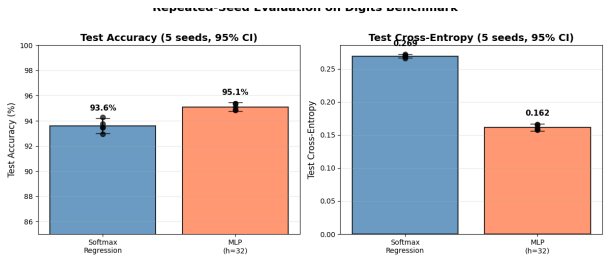


Figure 5: Repeated-seed evaluation comparing Softmax Regression and MLP on Digits. Individual seed results shown as black dots, bars show means, error bars show 95% CI.

This demonstrates that the hidden layer provides genuine representational benefit when the problem requires nonlinear separation.

4.3 Core comparison on the digits benchmark

On the digits benchmark, the MLP outperforms softmax regression on both metrics (Table 1, Figure 5). The accuracy improvement of 1.5 percentage points is statistically significant. More notably, cross-entropy is reduced by 40% (0.269 ⟶ 0.162), indicating substantially

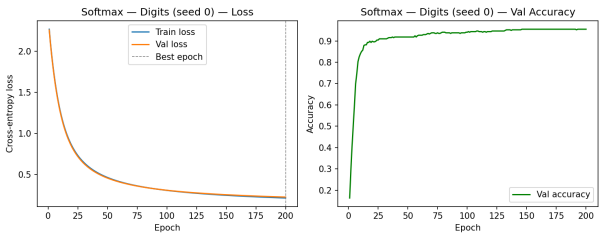


Figure 6: Training dynamics for softmax regression on Digits. Loss decreases smoothly, validation accuracy plateaus around epoch 150.

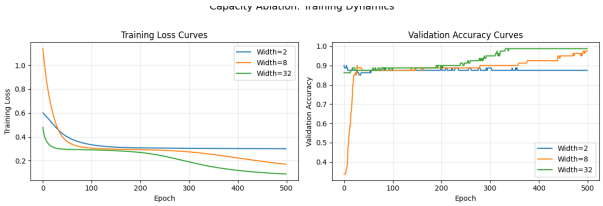


Figure 7: Training dynamics for capacity ablation on Moons. Larger width achieves lower loss and higher accuracy.

better-calibrated predictions. (We cann see the comparison visually in (Figure 5) bar chart).

4.4 Required capacity ablation on moons

The capacity ablation (Figure 4, Figure 7) reveals:

- **Width=2**: With only 2 hidden units, the network operates in a clear under-parameterized regime. Its limited representational capacity prevents it from modeling the highly nonlinear structure of the moons dataset. As a result, the learned decision boundary is nearly linear, and the training loss quickly plateaus at a relatively high value. The model achieves approximately 85% accuracy, comparable to softmax regression, indicating that it fails to capture the intrinsic curvature of the data.
- **Width=8**: Increasing the width to 8 significantly enhances the model's expressive power. The network is now capable of approximating a nonlinear decision boundary that better aligns with the data geometry. This is reflected in both improved accuracy (95%) and faster convergence of the training loss. The smoother and more consistent decrease in loss suggests that the optimization process benefits from the increased parameter space, allowing more effective gradient-based updates.
- **Width=32**: With 32 hidden units, the model enters a high-capacity regime. It achieves the lowest training loss and highest accuracy (97.5%), learning a smooth and well-fitted nonlinear boundary. Notably, there is no observable overfitting: the validation accuracy remains stable throughout training. This indicates that the additional capacity primarily improves representation and optimization rather than leading to memorization of the training data.

Beyond final accuracy, Figure 7 highlights important differences in training dynamics. Narrow networks (width = 2) exhibit early

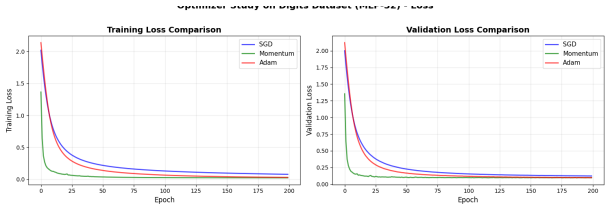


Figure 8: Loss curves for three optimizers on Digits (MLP-32). Momentum converges fastest.

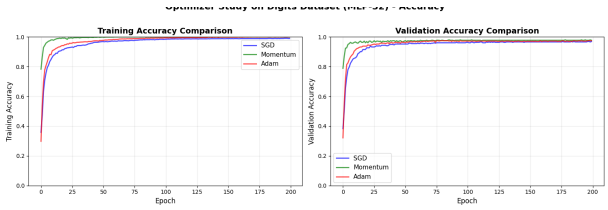


Figure 9: Accuracy curves for three optimizers on Digits. All three achieve similar final accuracy, but Momentum is most stable.

saturation of the loss, reflecting a limitation in representational capacity rather than optimization. In contrast, wider networks maintain a decreasing loss trajectory over a longer range of epochs, suggesting that increased width facilitates optimization by providing a richer set of descent directions in parameter space.

This ablation demonstrates the "capacity–problem complexity" tradeoff: models with insufficient capacity underfit and fail to represent the required nonlinear decision boundary, while sufficiently wide models can approximate the true data structure and achieve strong generalization. Importantly, the absence of overfitting in higher-capacity models suggests that, for this task, increasing width primarily reduces bias without significantly increasing variance.

4.5 Required optimizer study on digits

We compare SGD (lr=0.05), Momentum (lr=0.05, $\beta = 0.9$), and Adam (lr=0.001) on the MLP-32 model (Figure 8, Figure 9).

Convergence Behavior. The loss curves in Figure 8 reveal clear differences in optimization dynamics:

- **Momentum** achieves the fastest convergence, rapidly decreasing the loss within the first 25 epochs. This behavior arises because momentum accumulates past gradients, effectively accelerating updates along consistent descent directions and reducing oscillations.
- **Adam** converges more gradually in the early stages due to its smaller learning rate and adaptive scaling of gradients. However, it steadily approaches a similar final loss, benefiting from per-parameter step size adaptation.
- **SGD** exhibits the slowest convergence. Without momentum or adaptive scaling, updates are more sensitive to local curvature, resulting in smaller effective progress per iteration and requiring the full training schedule to approach optimal loss.

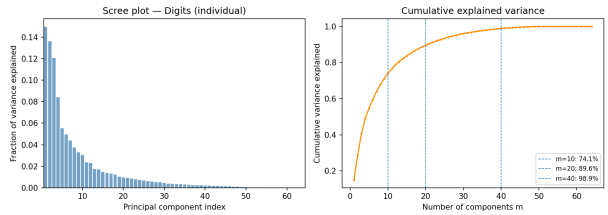


Figure 10: Scree plot for Digits dataset. Left: individual variance explained per component. Right: cumulative variance with key thresholds marked.

Accuracy and Generalization. Despite differences in convergence speed, Figure 9 shows that all three optimizers achieve similar final validation accuracy (approximately 97%). This indicates that the optimization landscape for this task is relatively well-conditioned, allowing different optimizers to reach comparable solutions given sufficient training time.

Stability and Training Dynamics. Momentum and Adam exhibit smoother and more stable training curves compared to SGD. In particular, momentum reduces high-frequency oscillations in the loss trajectory, leading to more consistent improvement across epochs. This stability is especially evident in the validation curves, where momentum achieves faster and more reliable convergence.

These results suggest that, for this problem, the choice of optimizer primarily influences the **speed and stability of convergence** rather than the final generalization performance.

4.6 Required failure-case analysis

Softmax achieves only 85% accuracy on the Moons dataset while the MLP achieves 98%.

On datasets like the Moons dataset, simple linear models such as softmax regression struggle, highlighting why more flexible models like multilayer perceptrons are often necessary.

Mechanism: Softmax regression computes $s(x) = Wx + b$, producing linear decision boundaries. The Moons dataset requires a curved separator. Mathematically, no choice of W and b can perfectly separate two interleaved crescents with a line. This is a fundamental limitation of the model class, not a training failure.

Lesson: Model complexity must match data complexity. When class boundaries are inherently nonlinear, a linear model will fail regardless of training duration or hyperparameter tuning.

5 Advanced Track

5.1 Track A: PCA/SVD and input geometry

PCA analysis reveals the intrinsic dimensionality of the digits dataset (Figure 10):

- $m = 10$: 74.1% cumulative variance
- $m = 20$: 89.6% cumulative variance
- $m = 40$: 98.9% cumulative variance

These results indicate that although the original data lies in a 64-dimensional space, most of the informative structure is concentrated in a significantly lower-dimensional subspace.

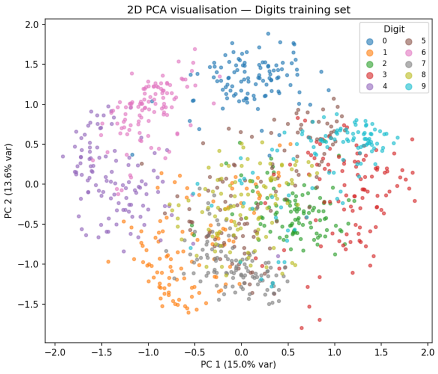


Figure 11: 2D PCA projection of Digits training set. PC1 and PC2 explain 28.6% of variance combined. Digit clusters are partially separable but heavily overlapped.

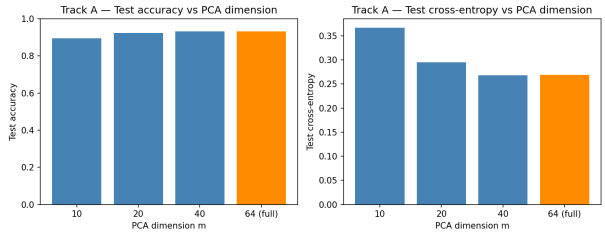


Figure 12: Softmax performance vs PCA dimension. Accuracy plateaus after 20 components; cross-entropy continues decreasing with more components.

The 2D PCA projection (Figure 11) shows partial cluster separation. Digits such as 0, 4, and 6 form relatively distinct clusters, while many other classes overlap substantially, confirming that the dataset is not perfectly linearly separable even after dimensionality reduction.

Effect of Dimensionality on Model Performance. The relationship between dimensionality and model performance is illustrated in (Figure 12):

- Test accuracy increases rapidly as the number of PCA components grows from 10 to 20
- Beyond 20 components, accuracy improvements become marginal (plateau effect)
- Cross-entropy loss continues to decrease slightly even after accuracy stabilizes

Interpretation: Additional dimensions may refine predicted probabilities without significantly improving classification decisions. This highlights diminishing returns in increasing dimensionality: after a certain threshold, new components primarily capture noise rather than meaningful signal.

Intrinsic Dimensionality and Linear Separability. PCA results suggest that the digits dataset has an effective dimensionality of approximately 20–40. In this reduced space, many class boundaries become approximately linear, explaining why softmax regression performs competitively despite being a linear model.

Residual overlap between classes indicates that perfect separation requires nonlinear decision boundaries, which is where neural networks provide an advantage.

Role of Nonlinearity in Feature Learning. While PCA provides a linear transformation of the data, neural networks learn nonlinear feature representations. This reshapes the input space to improve class separability.

Unlike PCA, which is unsupervised and variance-driven, neural networks learn task-specific features that directly optimize classification performance. As a result, even when PCA captures most of the variance, it may not capture the most discriminative features, explaining why the MLP achieves improved performance on the digits dataset.

Practical Trade-offs and Model Selection. From a practical standpoint, there is a trade-off between model complexity and performance:

- Linear models are computationally efficient and perform well when data is approximately linearly separable
- Nonlinear models (MLPs) provide additional flexibility at the cost of increased computation and risk of overfitting

In our project, the improvement of the MLP over softmax regression on the digits dataset is modest. This suggests that simpler models may often suffice when the data structure is favorable.

6 Discussion

When is the linear model sufficient? Our experiments demonstrate that linear softmax regression is geometrically sufficient when class boundaries are approximately linear in the input space. On the Gaussian dataset, the linear model achieved optimal 95% accuracy. On digits, the linear model achieved 93.6%, only 1.5 points below the MLP despite having far fewer parameters. This suggests many real-world problems may be effectively linear in high-dimensional feature spaces.

What representational change does the hidden layer provide? The hidden layer learns a nonlinear feature transformation that makes the problem more linearly separable. On moons, this manifests as a curved decision boundary impossible for any linear model. On digits, the improvement is more subtle: the MLP learns combinations of pixel patterns that better distinguish visually similar digits (e.g., 3 vs 8).

When does additional complexity fail to justify itself? On the Gaussian task, the MLP provided zero improvement over softmax regression across all capacity levels. The additional parameters only increased computational cost without improving accuracy. This illustrates the principle of parsimony: when a simple model suffices, prefer it.

What does the failure case teach? The under-capacity MLP (width=2) on moons shows that nonlinearity alone is insufficient; the network must have enough hidden units to represent the required decision boundary. This has practical implications: model capacity should be chosen based on problem complexity, not arbitrarily.

Value of repeated-seed evaluation. Single-seed evaluation can be misleading due to initialization variance. Our repeated-seed results show that the 1.5% accuracy gap between softmax and MLP is

robust across 5 seeds with stable confidence intervals. This provides statistical confidence that the MLP genuinely outperforms the linear model on digits, rather than benefiting from lucky initialization.

7 Limitations

While the empirical results provide useful insights into the comparative behavior of softmax regression and shallow MLPs, several important limitations must be acknowledged. Importantly, these are not only constraints on *what* was tested, but also on *why* the observed outcomes may arise.

- **Limited dataset scope and complexity**: We evaluated the models on only three relatively small and structured datasets (e.g., Linear Gaussian). While this controlled setting is useful for isolating specific behaviors (such as capacity effects), it inherently restricts the diversity of data distributions encountered. Modern machine learning models often exhibit behaviors that emerge only at scale (e.g., high-dimensional feature interactions, distributional shifts, or noisy labels). Simpler datasets may favor models that rely on linear separability or low-complexity decision boundaries, thereby potentially underestimating the advantages of more expressive architectures like MLPs. As a result, our findings may not transfer to real-world tasks such as image or text classification, where hierarchical feature extraction is critical.
- **Restricted architectural design**: The study focuses exclusively on shallow neural networks with a single hidden layer and tanh activation. The representational power and optimization dynamics of neural networks are highly sensitive to architectural choices. Deeper networks can learn hierarchical representations, and alternative activations (e.g., ReLU) can mitigate issues such as vanishing gradients and saturation inherent in tanh. By limiting the architecture, we may be observing behavior specific to this narrow design space rather than general properties of neural networks. Consequently, the relative performance gap between softmax regression and MLPs could shift significantly under deeper or modern architectures.
- **Lack of hyperparameter optimization**: We relied primarily on default or minimally tuned hyperparameters (e.g., learning rate, initialization, regularization strength). Optimization performance is highly sensitive to these choices, especially for models like softmax regression that may require careful scaling or learning rate schedules to reach optimal solutions. Without systematic tuning (e.g., grid search or Bayesian optimization), it is unclear whether the observed performance differences reflect intrinsic model limitations or suboptimal training configurations. In particular, softmax regression might appear weaker simply because it was not trained under its most favorable conditions.
- **Fixed regularization strategy**: The experiments employ a uniform L2 regularization scheme across all models. Different models often benefit from different forms of regularization. For example, neural networks may respond better to dropout or early stopping, while linear models may benefit from stronger weight decay or sparsity-inducing penalties (e.g., L1). By enforcing a single regularization strategy, we may inadvertently bias the comparison, either constraining one model more than necessary or failing to prevent overfitting in another. This limits our ability to draw fair conclusions about their true generalization capabilities.
- **Optimization dynamics and training time horizon**: All models were trained for a fixed number of epochs with the same optimization procedure. Different models converge at different rates and may require different training schedules to fully realize their potential. For instance, wider networks often converge faster (as observed in Figure 13), while smaller models may require longer training or adaptive learning rate schedules. Fixing the training horizon may therefore advantage certain models over others, conflating optimization efficiency with final performance.
- **Simplified evaluation metrics**: The study primarily evaluates performance using training loss and validation accuracy. These metrics do not capture other important aspects such as calibration, robustness to noise, or sensitivity to distribution shifts. Two models with similar accuracy may differ significantly in confidence estimation or stability. By focusing on a narrow set of metrics, we may overlook nuanced but practically important differences between softmax regression and MLPs.

Overall, these limitations highlight that our conclusions are tightly coupled to the controlled experimental setting. The observed advantages of increased capacity and nonlinear modeling should therefore be interpreted as *context-dependent* rather than universal.

References

https://medium.com/@preethithakur/softmax-regression-93808c02e6ac-Softmax regression

https://share.google/nnkYWO9F15lSC7He0 -Softmax regression

https://www.geeksforgeeks.org/machine-learning/what-is-cross-entropy-loss-function/ -Cross-entropy loss

https://share.google/pfaoLWAQgl6CBM393 -Principal Component Analysis

https://www.geeksforgeeks.org/machine-learning/linear-regression-vs-neural-networks-understanding-key-differences/ -Linear regression vs. NN

https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/ -Linear Regression

https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/ -NN

https://www.geeksforgeeks.org/deep-learning/layers-in-artificial-neural-networks-ann/ -Layers in NN

https://www.geeksforgeeks.org/deep-learning/xavier-initialization/ -Xavier initialization lecture materials provided during the course

A Supplementary Training Curves

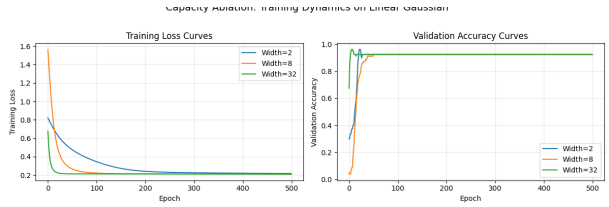


Figure 13: Training dynamics for capacity ablation on Linear Gaussian dataset.

Figure 13 illustrates the training dynamics under a capacity ablation study on the Linear Gaussian dataset, where model width is varied (Width = 2, 8, 32). The left plot shows the training loss as a function of epochs, while the right plot presents the corresponding validation accuracy.

From the training loss curves, we observe that models with larger width converge significantly faster and achieve lower loss values earlier in training. In particular, the Width=32 model rapidly reaches a near-optimal loss within the first few epochs, indicating higher representational capacity and more efficient optimization. In contrast, the Width=2 model exhibits slower convergence and maintains a higher loss for a longer duration, suggesting limited capacity to fit the data efficiently.

The validation accuracy curves further reinforce this observation. Wider models (Width=8 and Width=32) quickly achieve high accuracy (above 90%) and stabilize early, demonstrating strong generalization performance. The Width=2 model, however, shows a slower increase in accuracy and slightly lower final performance, indicating that insufficient capacity can hinder both learning speed and generalization.

B Contribution Statement

- **Gulnar Babazade**: Implemented softmax regression, sanity checks, and experiments on synthetic datasets.
- **Nadir Eskerov**: Implemented MLP, backpropagation, optimizers, and capacity ablation studies.
- **Narmina Ibrahimova**: Implemented PCA analysis (Track A), repeated-seed evaluation.
- **Evez Huseynov** : Implemented training for MLP, backward for mlp ,optimizers study on digits dataset.